

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Mock Interview

Duration: 1hr

Code of Conduct:

I Vignesh S / 192325002 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vignesh S

MOCK INTERVIEW — CANDIDATE RESPONSES

Candidate: Vignesh S (192325002). The following are my spoken responses to each interview question.

Q1. A machine learning engineer applies Reinforcement Learning to solve sequential decision-making problems. Explain how an agent learns optimal behavior through interaction with the environment.

Answer: Reinforcement Learning is essentially learning by trial and error. The agent observes the current state of the environment, takes an action, and in return receives a reward and a new state. Its objective is not to maximise the immediate reward but the cumulative discounted reward over the long run. To do that, it estimates value — how good a state or an action is in terms of expected future return — and gradually shifts its policy toward the actions with higher value. Early in training it explores to gather information, and as its estimates become reliable it exploits the best-known actions. The mathematical backbone is the Bellman equation, which says the value of a state equals the immediate reward plus the discounted value of the next state; learning rules such as temporal-difference updates or policy gradients use this to improve the policy step by step. A simple example: a robot in a maze that gets minus one per step and plus ten at the goal will, after enough episodes, learn the shortest path, because that trajectory yields the highest total return. So optimal behaviour emerges naturally from repeatedly interacting, receiving feedback, and updating.

Q2. If you are asked to design a Reinforcement Learning solution for a real-time system such as ride-sharing or delivery optimization, explain how you would formulate the problem using a Markov Decision Process.

Answer: I would begin by defining the five MDP components clearly. The state has to capture everything the agent needs to decide well — driver locations and availability, pending requests, traffic, time of day, and the demand-supply balance across zones. The actions are the decisions: which driver to assign to a request, whether to reposition an idle driver, or what price to set. The transition function is the city's dynamics, and it is stochastic because demand and traffic are uncertain. The reward encodes the business goal — positive for completed trips and short waiting or pickup times, negative for cancellations, idle time, and long detours. The discount factor matters here because assigning a driver now changes availability later, so we must value the long term. I would make the state rich enough to respect the Markov property, though in reality it is partially observable, so I would treat it as a POMDP and use function approximation given the huge state space. Finally I would pick an algorithm suited to that scale — an actor-critic or policy-based method — and train first in a simulator before deploying online.

Q3. Compare model-free and model-based Reinforcement Learning approaches. Explain their working principles and how they handle uncertainty.

Answer: Model-free RL learns directly from experience without ever building a model of the environment. It learns either value functions, as in Q-learning or SARSA, or policies directly, as in policy-gradient methods, purely from sampled transitions. Model-based RL instead first learns the environment's dynamics and reward, then plans with that model — for instance by simulating rollouts or running value iteration. The trade-off is central: model-free is simpler and unbiased but very sample-hungry, whereas model-based is far more sample-efficient because it reuses each real interaction by planning in the learned model, but it is vulnerable to model error — if the model is wrong, planning is misled. On uncertainty, model-free handles it implicitly by averaging over many samples, so its estimates converge to true expected returns. Model-based methods handle it explicitly, often using probabilistic models or ensembles that represent how confident the model is, so the planner can stay cautious where the model is unreliable — PETS is a good example. In short, model-free trusts the data, model-based trusts a model but must quantify how far to trust it.

Q4. A recommendation system using Reinforcement Learning is underperforming. Explain how reward sparsity and hyperparameter tuning can be diagnosed and improved.

Answer: I would diagnose along two lines. First, reward sparsity: if the only reward is a rare event like a purchase or long-term retention, the agent receives almost no learning signal, so training stalls. I would measure how often non-zero rewards actually occur. To fix it I would use reward shaping — adding intermediate signals such as clicks, dwell time, or add-to-cart — ideally potential-based shaping so the optimal policy is preserved, and I might add curiosity-driven exploration to densify the signal. Second, hyperparameters: I would examine the learning rate, because too high makes value estimates oscillate and too low makes learning crawl; the exploration rate and its decay; the discount factor, since too low makes the agent myopic and ignore long-term engagement; and the replay and batch settings. My method is empirical — plot the learning curve and the reward distribution, run a small grid or Bayesian search over the few most sensitive hyperparameters, and watch for divergence or premature convergence. Practically I fix reward sparsity first, because no amount of tuning helps if the signal simply is not there.

Q5. If you are implementing an RL-based game-playing AI, describe how you would design the learning pipeline and ensure training stability.

Answer: I would build the pipeline in stages. First the environment interface — the game emulator that provides states, usually raw frames, a defined action set, and a reward such as the change in score or a win-loss signal. Then preprocessing like frame stacking and normalisation, feeding a convolutional network with a value or policy head. I would choose the algorithm to match the action space — DQN for discrete actions, or PPO or A3C for policy-based control. For learning I would use experience replay to decorrelate samples and a target network to stabilise the bootstrapped targets. For stability specifically, I would clip rewards and gradients, update the target network slowly, tune the learning rate carefully, and in PPO rely on the clipped objective to prevent destructive updates. I would anneal exploration over time, monitor the loss and the average return, and evaluate on unseen game seeds to catch overfitting, while logging and seeding everything for reproducibility. If training diverges, the usual suspects are a too-high learning rate, too-frequent target updates, or unclipped rewards, so those are the first knobs I check.

Q6. A company wants to use Reinforcement Learning for dynamic pricing. Explain how the agent learns from interaction and improves pricing decisions over time.

Answer: The agent treats pricing as a sequence of decisions. The state is the market context — current demand, inventory, competitor prices, time, and customer segment — and the action is the price or discount level. When it sets a price, customers respond by buying or not, and the reward is the realised profit. The key point is that the agent never sees the demand curve directly, so it has to learn it: it explores by occasionally testing different price points, observes how demand responds, and then exploits the prices that maximise long-term profit. Over many interactions its value estimates for each price in each context sharpen, and pricing steadily improves. I would emphasise the exploration-exploitation balance, because testing prices costs some short-term revenue but is essential for learning elasticity, and I would bound exploration so we never post absurd prices. I would also let the discount factor capture longer-term effects — a discount today can build loyalty tomorrow. If pricing decisions are largely independent I would start with a contextual bandit; if inventory or sequential effects are strong, I would move to full RL.

Q7. If an RL model is not converging properly, explain how you would identify and fix issues related to learning rate and exploration strategy.

Answer: I treat non-convergence as a systematic diagnosis. For the learning rate, if the reward curve oscillates wildly or the loss blows up, the rate is too high, so I lower it, switch to an adaptive optimiser like Adam, and possibly add a decay schedule; if learning is flat and painfully slow, it may be too low, so I raise it. For exploration, if the agent settles early on a mediocre policy, it is exploiting too soon — I increase epsilon or slow its decay, switch to a smarter scheme like softmax or UCB, or add entropy regularisation in policy methods to keep the policy stochastic. If it never settles at all, exploration may be too high for too long, so I decay it faster toward the end. I also check the basics that masquerade as non-convergence — reward scaling, the discount factor, and whether the state representation is even Markov. Throughout, I change one thing at a time, watch the learning curve and the value estimates, and confirm the fix on a simplified version of the task before scaling back up.

Q8. A robotics system uses Reinforcement Learning for navigation. Explain how the environment and agent interaction leads to optimal path learning.

Answer: Here the robot is the agent and the physical space, or its simulation, is the environment. The state is the robot's pose and sensor readings — position, orientation, nearby obstacles, and distance to the goal — and the actions are motion commands. On each interaction the robot moves, the environment returns a new state and a reward, typically a small negative per step to encourage efficiency, a large positive at the goal, and penalties for collisions. Through many episodes the robot estimates the value of states and actions, so it learns which moves lead toward the goal fastest while staying safe. The optimal path emerges because that trajectory maximises cumulative reward — fewest steps and no collisions. Early on it explores widely, and as value information propagates back from the goal through temporal-difference updates, the greedy policy increasingly follows the shortest safe route. In practice I would train mostly in simulation for safety and sample efficiency, use a shaped reward like distance-to-goal to speed learning, enforce safety constraints so exploration never damages the robot, and then handle the sim-to-real transfer carefully.

Q9. If you are asked to improve an RL model with delayed rewards, explain techniques to handle credit assignment problems.

Answer: The credit assignment problem is deciding which earlier actions deserve credit for a reward that arrives much later. Several techniques help. Discounting and the return itself already spread credit backward, and temporal-difference learning bootstraps so value propagates one step at a time. To speed that up I would use eligibility traces, or TD-lambda, which assign credit to a decaying trace of recently visited states and bridge the gap between one-step TD and Monte Carlo. Reward shaping, especially potential-based shaping, adds intermediate signals without changing the optimal policy. Using n-step returns or Monte Carlo methods feeds distant rewards back faster over longer horizons. For very long horizons, hierarchical RL breaks the task into sub-goals so credit is assigned within short sub-episodes, and hindsight experience replay relabels goals so even failed trajectories still teach something. In deep RL, advantage estimators like GAE balance bias and variance when propagating delayed reward. I would choose based on the horizon — eligibility traces and GAE for moderate delays, and hierarchy for very long ones.

Q10. A financial firm uses Reinforcement Learning for trading decisions. Explain how uncertainty and risk can be handled in the learning process.

Answer: Markets are noisy, non-stationary, and partially observable, so I would design for uncertainty explicitly. First, the reward: instead of raw profit I would use a risk-adjusted objective such as the Sharpe ratio, or profit penalised by volatility and drawdown, so the agent does not chase high-variance bets, and I would include transaction costs so it does not over-trade. Second, I would treat the problem as partially observable and give the agent a rich state

— recent prices, volatility, technical indicators, and current position — possibly with a recurrent policy to capture hidden market regimes. Third, to handle model and market uncertainty I would use ensembles or distributional RL, which learns the whole distribution of returns rather than just the mean, allowing me to act on risk directly through objectives like conditional value-at-risk. Fourth, I would layer hard risk limits — position caps and stop-losses — on top of the learned policy as safety constraints, because a learned agent should never be free to blow up the account. Finally, since markets shift, I would keep learning online and validate rigorously on out-of-sample and stress periods. The theme is: optimise risk-adjusted return, quantify uncertainty, and constrain the downside.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately